2. Hafta

Fonksiyonlar ve Bellek Modeli

◎ Öğrenme Hedefleri

Bu bölümü tamamladığınızda:

- Aynı ada sahip farklı imzalı fonksiyonlar (aşırı yükleme) ve varsayılan parametreler tanımlayabileceksiniz;
- Bir parametreyi değere göre, referansla, `const` referansla veya işaretçiyle geçmek arasındaki farkları açıklayıp doğru olanı seçebileceksiniz;
- Yığın (otomatik ömür) ile öbek (dinamik ömür) arasındaki farkı, `new`/`delete` ve `nullptr` kullanımını uygulayabileceksiniz;
- Bellek sızıntısı ve sarkan işaretçi hatalarını tanıyıp nedenini açıklayabileceksiniz;
- RAII fikrini (kaynağı bir nesnenin ömrüne bağlama) ilk bakış düzeyinde uygulayıp elle `new`/`delete` yönetiminin risklerini karşılaştırabileceksiniz.

Geçen hafta C++'ın temel sözdizimini ve zihniyetini gördünüz; referansları (`int&`) ve `.hpp`/`.cpp` + CMake iskeletini tanıdınız. Bu hafta o referans bilgisini derinleştirip fonksiyonların kendisini — aşırı yükleme, varsayılan parametre — ve C++'ın bellek modelini işliyorsunuz. Kitaplık teması henüz başlamadı — Hafta 3'te `Kitap` sınıfıyla başlayacak; bu hafta boyunca Hafta 1'den tanıdığınız `geometri::Nokta` aynen taşınır, yanına RAII'ye ilk bakış için tasarlanmış yeni, küçük bir tip — `araclar::IntTampon` — eklenir.

2.1 Fonksiyonlar: Aşırı Yükleme ve Varsayılan Parametreler

1 • Ön-öğretim **📣 Neden Önemli?**

C'de bir şeklin alanını hesaplarken her tür için ayrı bir ad uydurmak zorundaydınız: `alanKare`, `alanDikdortgen`, `alanDaire`. Aynı işi yapan üç fonksiyon, üç farklı ad demektir; çağıran taraf her seferinde "bunun için doğru ad neydi?" diye hatırlamak zorunda kalır. Oysa yaptığınız iş hep aynıdır: alan hesaplamak. Bu ünite, tek bir anlamlı adı (`alan`) verdiğiniz argümanlara göre otomatik seçtiren yolu açar ve sık kullandığınız bir varsayılanı her seferinde yazmaktan sizi

kurtarır.

1 • Ön-öğretim **Ön Bilgi**

Bu üniteye girmeden önce şunları hatırlayın: Hafta 1'den ad alanları, `const` ve referans girişi; C'den taşıdığınız fonksiyon tanımı/çağrısı ve "her fonksiyonun benzersiz ada sahip olması" kuralı. Bu son kural tam da bu ünitenin gevşettiği varsayımdır; aşırı yüklemede derleyicinin sürümleri ayırt etmek için baktığı parametre listesi ve tip kavramlarının taze olması gerekir.

Geçen hafta C++'ın temel sözdizimini, ad alanlarını ve referansları gördünüz; fonksiyon tanımlayıp çağırmayı ise zaten C'den biliyorsunuz — bir ad, bir parametre listesi, bir gövde. Bu hafta o bildik fonksiyon kavramını iki yönde genişletiyorsunuz: aynı işin birden çok "biçimini" tek bir ada bağlamak (aşırı yükleme) ve bir parametreye öntanımlı bir değer vererek çağıranın onu bazen atlayabilmesini sağlamak (varsayılan parametre). İkisi de C'de karşılığı olmayan, saf C++ araçlarıdır.

2.1.1 Aşırı yükleme: aynı ad, farklı imza

C'de bir fonksiyon adı tüm programda *benzersiz* olmak zorundadır — iki farklı yerde aynı adla iki fonksiyon tanımlarsanız, bağlayıcı bir çakışma hatası verir. Bunun pratikte ne anlama geldiğini somut bir senaryoyla görün: bir karenin, bir dikdörtgenin ve bir dairenin alanını hesaplayan üç fonksiyon yazmak istediğinizi düşünün. C'de bunların üçüne de `alan` adını veremezsiniz; `alanKare`, `alanDikdortgen`, `alanDaire` gibi üç ayrı, birbirinden yapay biçimde farklılaştırılmış ad uydurmanız gerekir — oysa kavramsal olarak üçü de aynı işi yapar: "bana bir şeklin ölçülerini ver, sana alanını hesaplayayım." Adların farklı olması, kodu okuyan kişiye gereksiz bir bilgi yükü bindirir: hangi adın hangi şekle karşılık geldiğini ayrıca ezberlemeniz gerekir.

C++ bu sorunu farklı bir kuralla çözer:

Tanım: Aşırı yükleme

Aşırı yükleme (*function overloading*), aynı adı taşıyan birden çok fonksiyonu — parametre listeleri (sayı ve/veya tip bakımından) birbirinden farklı olmak koşuluyla — aynı kapsamda tanımlama olanağıdır. Bir çağrı yapıldığında derleyici, çağrıda verilen argümanların sayısına ve tipine bakarak hangi sürümün kastedildiğine *derleme anında* karar verir.

Burada kritik sözcük "parametre listesi"dir — derleyicinin iki fonksiyonu birbirinden ayırt etmek için baktığı şey budur, ad tek başına yeterli değildir:

Tanım: İmza

Bir fonksiyonun *imzası* (*signature*), onu diğer fonksiyonlardan ayırt eden ad ve parametre listesi (parametre sayısı ve tipleri) bütünüdür. **Dönüş tipi imzanın bir parçası değildir** — iki fonksiyon yalnızca dönüş tipiyle farklılaşıyorsa, derleyici bunları aynı imza olarak görür ve ikisini bir arada tanımlamanıza izin vermez.

Bu tanımı somut kodda görün:

```
5 int alan(int kenar) {
6     return kenar * kenar;
```

```
}

int alan(int taban, int yukseklik) {
    return taban * yukseklik;
}

double alan(double yaricap) {
    return 3.14159 * yaricap * yaricap;
}
```

Üç `alan` fonksiyonunun da adı aynıdır, ama imzaları üç farklı biçimde ayrışır: birincisi tek bir `int` parametre alır (kenar uzunluğu), ikincisi iki `int` parametre alır (taban ve yükseklik), üçüncüsü tek bir `double` parametre alır (yarıçap). Bir çağrı yaptığınızda derleyicinin izlediği mantığı adım adım düşünün: `alan(4)` yazdığınızda derleyici argüman listesine bakar — tek bir `int` değer — ve bu, yalnızca birinci sürümün imzasıyla eşleşir, o çağrılır. `alan(3, 5)` yazdığınızda iki `int` değer vardır, yalnızca ikinci sürümün imzasıyla eşleşir. `alan(2.0)` yazdığınızda tek bir `double` değer vardır, yalnızca üçüncü sürümle eşleşir. Derleyici bu eşleştirmeyi programınız hiç çalışmadan, yalnızca kaynak kodu okuyarak yapar — çalışma zamanında "hangi `alan`'ı çağıracağım" diye bir belirsizlik ya da ek maliyet söz konusu değildir, karar tamamen derleme anında verilir ve kesinleşir.

Analoji

Aşırı yüklemeyi gündelik dildeki bir fiilin farklı bağlamlarda farklı bir eylemi karşılamasına benzetebilirsiniz. "Aç" fiilini düşünün: "kapıyı aç", "dosyayı aç", "mağazayı aç" — üçünde de aynı sözcüğü kullanırsınız, ama fiilin yanında hangi ismin (parametrenin) geçtiğine bakarak dinleyici hangi eylemi kastettiğinizi otomatik çıkarır; kapıyı elinizle ittirirsiniz, dosyayı bir programda tıklarsınız, mağazayı vitrin panjurunu kaldırıp kapıdaki tabelayı çevirirsiniz — üçü bambaşka eylemlerdir ama tek bir fiil adı hepsini taşır. Derleyici de tam olarak dinleyicinin yaptığı şeyi yapar: `alan(...)` çağrısını gördüğünde, yanındaki "parametreye" — argümanların sayı ve tipine — bakarak hangi `alan` sürümünün kastedildiğini otomatik çözer. Siz kodu yazarken hangi "alan"ı kastettiğinizi ayrıca belirtmek zorunda kalmazsınız; tıpkı "aç" derken kapı mı dosya mı olduğunu ayrı bir kelimeyle açıklamanıza gerek kalmaması gibi.

İmza tanımındaki "dönüş tipi imzanın bir parçası değildir" uyarısını ciddiye alın — bu, sık düşülen bir tuzağın kaynağıdır:

Ortak Hata / Tuzak

Yalnızca dönüş tipiyle birbirinden ayrılan iki fonksiyon tanımlamaya çalışmak derlenmez:

```
int alan(int kenar); // ilk tanim
double alan(int kenar); // DERLEME HATASI: ayni imza (int), farkli
    donus tipi
```

Derleyici bir çağrıyı — örneğin `alan(4)` — yalnızca argüman listesine bakarak çözer; sonucun nereye atanacağını (dolayısıyla dönüş tipini) çoğu zaman çağrı anında henüz "bilmez" — `alan(4)`'ün sonucunu bir değişkene hiç atamadan da kullanabilirsiniz (ör. doğrudan `std::cout` içine verebilirsiniz). Eğer iki `alan(int)` sürümü bir arada dursaydı, derleyici `alan(4)` çağrısında hangisinin çalışacağını — `int` mi `double` mü dönecek — belirleyemezdi; bu

yüzden C++ böyle bir belirsizliği daha en baştan, derleme hatasıyla engeller.

2.1.2 Varsayılan parametreler

Aşırı yükleme aynı işin *farklı* parametre kombinasyonlarını kapsıyordu; varsayılan parametre ise tek bir fonksiyonun bir parametresini isteğe bağlı hâle getirmenizi sağlar. Şu fonksiyonu düşünün: bir sayının bir kuvvetini hesaplayan `guc(taban, us)`. Kod tabanınızda sayıların *karesini* almak (`us = 2`) çok daha sık istenen bir işlemse, her çağrıda `guc(taban, 2)` yazıp `2`'yi tekrar tekrar elle geçirmek gereksiz bir tekrar ve unutulmaya açık bir ayrıntıdır. C++ bu tekrarı ortadan kaldıran bir araç sunar:

Tanım: Varsayılan parametre

Bir *varsayılan parametre* (*default parameter*), fonksiyon bildiriminde `tip ad = deger` biçiminde bir öntanımlı değer alan parametredir. Çağıran taraf bu parametre için bir değer belirtmezse, derleyici bildirimdeki değeri otomatik kullanır; çağıran taraf isterse kendi değerini vererek varsayılanı geçersiz kılabilir.

Analoji

Bunu bir kahve dükkânındaki sipariş alışkanlığına benzetebilirsiniz. "Bir kahve" dediğinizde boy belirtmezseniz size dükkânın *varsayılan* boyu — genelde orta boy — gelir; siz herhangi bir şeyi "eksik" bırakmış olmazsınız, dükkân sizin adınıza makul bir öntanımlı seçim yapmıştır. İsterseniz "büyük kahve" diyerek bu varsayılanı açıkça geçersiz kılarsınız. Fonksiyon çağrısında bir parametreyi atlamak da tam olarak budur: "değeri siz seçin" demezsiniz, yalnızca dile önceden tanımlanmış makul bir seçimi kullanmasını söylersiniz.

Bu deseni somut kodda görün:

```
19 long guc(int taban, int us = 2) {
20    long sonuc = 1;
21    for (int i = 0; i < us; ++i) {
22       sonuc *= taban;
23    }
24    return sonuc;
25 }
```

`guc(int taban, int us = 2)` bildirimindeki `= 2` ifadesi, `us` parametresinin varsayılan değeridir. İki çağrıyı yan yana izleyin: `guc(3)` yazdığınızda yalnızca `taban` argümanını verirsiniz, derleyici `us` için otomatik olarak `2`'yi kullanır — sonuç $3^2 = 9$. `guc(3, 3)` yazdığınızda ise `us`'u açıkça `3` olarak belirtirsiniz, varsayılan devre dışı kalır — sonuç $3^3 = 27$. Dikkat edin: fonksiyonun gövdesi tek bir tanedir, iki ayrı `guc` sürümü yoktur — aşırı yüklemeden farklı olarak burada derleyici birden çok fonksiyon arasında seçim yapmaz, tek fonksiyonu eksik argümanla *tamamlar*.

Varsayılan değerler yalnızca bildirimin *sonundaki* parametrelere verilebilir — yani varsayılanı olan bir parametreden sonra varsayılansız bir parametre gelemez. Bunun nedenini çağrı tarafından düşünün: `f(int a, int b = 5, int c)` gibi bir bildirim geçerli olsaydı, `f(1, 3)` çağrısı belirsiz olurdu — derleyici `3`'ün `b`'ye mi yoksa `c`'ye mi ait olduğunu bilemezdi, çünkü argümanlar sırayla eşleşir ve aradaki bir

parametre atlanamaz. Varsayılanlı parametreleri her zaman listenin sonuna — en sağdaki parametrelerden başlayarak — yerleştirmeniz gerekir.

İpucu

Aşırı yükleme ile varsayılan parametreyi aynı anda, dikkatsizce kullanmak belirsizliğe (*ambiguity*) yol açabilir. Örneğin hem `void f(int a)` hem `void f(int a, int b = 0)` bir arada tanımlıysa, `f(5)` çağrısı iki sürümle de eşleşebilir — derleyici hangisini seçeceğini bilemez ve derleme hatası verir. Bu haftalık kapsamda bu tuzağın *var olduğunu* bilmeniz yeterli; iki aracı bir arada kullanırken parametre listelerinin gerçekten ayrışık kaldığından emin olun.

Bu ünitede gördüğünüz fikir kısaca şudur: aşırı yükleme aynı adı taşıyan, imzaları (ad + parametre listesi, dönüş tipi hariç) birbirinden ayrışık birden çok fonksiyon tanımlamanıza izin verir — derleyici doğru sürümü çağrı anındaki argümanlara bakarak seçer; varsayılan parametre ise tek bir fonksiyonun bir parametresini isteğe bağlı kılarak çağıranın onu atlayabilmesini sağlar. Gelecek ünitede bu fonksiyonların parametrelerini — Hafta 1'den tanıdığınız `geometri::Nokta` üzerinde — dört farklı biçimde nasıl geçirebileceğinizi derinlemesine göreceksiniz.

3 • Katılım — Şimdi Siz Deneyin

Aynı çeviri biriminde şu iki bildirim birlikte bulunabilir mi: `int hesapla(int a, int b);` ve `double hesapla(int a, int b);`? Peki `int hesapla(int a);` ile `int hesapla(int a, int b);` birlikte bulunabilir mi?

Geribildirim. İlk çift bulunamaz. Bir fonksiyonu ayırt eden şey ad ile parametre listesidir (imza); dönüş tipi imzaya dahil *değildir*. İki bildirim de `hesapla(int, int)` imzasına sahip olduğundan, derleyici bunları aynı fonksiyonun çelişkili iki tanımı olarak görür ve "yeniden tanımlama" hatası verir. İkinci çift bulunabilir: parametre sayıları farklıdır (bir ve iki parametre), dolayısıyla imzalar farklıdır — geçerli bir aşırı yüklemedir; derleyici çağrıyı verilen argüman listesine bakarak çözer.

Kendinizi Sınayın 2.1

Yalnızca dönüş tipiyle birbirinden ayrılan (parametre listeleri tıpatıp aynı olan) iki fonksiyon neden derlenmez? Yanıtınızda imza kavramını kullanın.

5 • İzleme — Aktarım — Başka Nerede?

Aynı adın argümana göre farklı anlam taşıması yalnız `alan` örneğine özgü değildir. İleride göreceğiniz Kitaplık projesinde bir kitabı hem ISBN'e (`bul(std::string isbn)`) hem yayın yılına (`bul(int yil)`) göre arayan iki `bul` işlevini aynı anlamlı ad altında toplayıp argümana göre ayırabilirsiniz. Varsayılan parametre de her yerde işe yarar: bir raf kapasitesini belirtmediğinizde makul bir öntanımlı değerin devreye girmesi gibi. "Bu iki işlev aynı işi mi yapıyor, yalnız girdisi mi farklı?" sorusu, ne zaman aşırı yükleme kullanacağınızı doğrudan söyler.

2.2 Parametre Geçme: Değer, Referans, `const` Referans, İşaretçi

1 • Ön-öğretim — Neden Önemli?

Bir fonksiyona bir `Nokta` verdiniz, fonksiyon onu "öteledi", ama `main`'e döndüğünüzde `Nokta` hiç değişmemiş — ya da tam tersi, hiç beklemediğiniz hâlde değişmiş. Bu şaşırtıcı davranışın tek nedeni, parametreyi hangi biçimde geçtiğinizdir: kopya mı verdiniz, nesnenin kendisini mi, yoksa adresini mi? Büyük bir nesneyi her çağrıda gereksiz yere kopyalamak da sessiz bir maliyettir. Bu ünite dört geçme biçimini ayırarak "değişsin mi, kopyalansın mı" kararını bilinçli vermenizi sağlar.

1 • Ön-öğretim — Ön Bilgi

Bu üniteye girmeden önce Hafta 1'in referans (`int&` — "takma ad") ve `const` kavramlarını, C'den bilinen işaretçiyi (`T*`) ve çağrı tarafında adres almayı (`&`) hatırlayın. Hafta 1'in `geometri::Nokta` değer tipini bu ünitede aynen kullanırsınız; referansın "kopya değil, ikinci bir isim" olduğu fikri buranın çekirdeğidir.

Geçen hafta `int&` ile referansları tanıdınız: bir referansın var olan bir değişkene ikinci bir isim olduğunu, boş olamayacağını, yeniden bağlanamayacağını gördünüz. O tanıtım tek bir örnek üzerinden (`ikiyeKatla(int&)`) geçti; bu ünitede aynı fikri, Hafta 1'den tanıdığınız `geometri::Nokta` üzerinde, dört farklı parametre geçme biçimini yan yana koyarak derinleştiriyorsunuz: değere göre, referansla, `const` referansla ve işaretçiyle geçme. Dördü de "bir fonksiyona veri nasıl ulaştırılır" sorusuna farklı bir cevap verir; hangisini ne zaman seçeceğinizi bu ünitenin sonunda net bir karar tablosuyla göreceksiniz.

2.2.1 Değere göre geçme: kopya alırsınız

Tanım: Değere göre geçme

Değere göre geçme (*pass by value*), bir parametrenin, çağrı anında argümanın bir *kopyasının* oluşturulup fonksiyona geçirilmesi biçimidir. Fonksiyon gövdesi içindeki her değişiklik yalnız bu kopya üzerinde gerçekleşir; çağırandaki orijinal nesne hiçbir zaman etkilenmez.

Analoji

Değere göre geçmeyi bir belgenin fotokopisini vermeye benzetebilirsiniz. Karşı tarafa orijinal belgeyi değil, bir fotokopisini verirsiniz; fotokopinin üzerine ne yazarsa yazsın, ne çizerse çizsin, elinizdeki orijinal belge bundan hiç etkilenmez — çünkü ortada artık iki ayrı kağıt vardır.

Bu davranışı somut kodda görün:

```
8  void oteleDeger(Nokta n) {
9      n.x += 10.0;
10     n.y += 10.0;
11 }
```

`oteleDeger(Nokta n)` imzasında `&` işareti *yoktur* — bu, `n`'nin çağırandaki nesnenin bir kopyası olduğu anlamına gelir. Fonksiyon çağrıldığı anda derleyici çağırandaki `Nokta` nesnesinin `x` ve `y` alanlarını, `n` adlı tamamen ayrı, yeni bir `Nokta` nesnesine kopyalar. Gövdedeki `n.x += 10.0;` satırı bu kopyayı değiştirir — ama fonksiyon geri döndüğünde bu kopya (ve üzerindeki değişiklik) birlikte yok olur; çağırandaki orijinal nesneye hiçbir zaman dokunulmamıştır.

Bu geçme biçiminin bir bedeli vardır: kopyalama işi ücretsiz değildir. Nesne küçükse (iki `double`'dan oluşan `Nokta` gibi) bu maliyet önemsizdir; ama nesne büyükse — örneğin yüzlerce elemanlı bir liste ya da uzun bir metin tutuyorsa — her çağrıda tam bir kopya oluşturmak gereksiz zaman ve bellek harcar.

2.2.2 Referansla geçme: takma ad verirsiniz

Geçen haftaki referans tanımını hatırlayın: bir referans, var olan bir değişkene verilen ikinci bir isimdir, ayrı bir kopya değildir. Bunu fonksiyon parametresine uyguladığınızda ortaya çıkan biçim şudur:

Tanım: Referansla geçme

Referansla geçme (*pass by reference*, `T&`), bir parametrenin çağırandaki nesneye doğrudan bir takma ad olarak bağlanmasıdır. Kopya oluşmaz — fonksiyon içindeki parametre ile çağırandaki nesne, bellekte aynı yeri gösteren tek bir varlıktır; bu yüzden fonksiyon gövdesindeki her değişiklik doğrudan çağırana yansır.

Analoji

Referansla geçmeyi, fotokopi yerine belgenin orijinalini elden vermeye benzetebilirsiniz — Hafta 1'deki lakap benzetmesinin bir fonksiyon çağrısına uzantısıdır. Karşı taraf artık kendi fotokopisiyle değil, belgenin *kendisiyle* çalışır; üzerine yaptığı her değişiklik kalıcıdır, çünkü ortada tek bir belge vardır, iki ayrı kopya yoktur.

```
15 void oteleReferans(Nokta& n) {
16     n.x += 10.0;
17     n.y += 10.0;
18 }
```

`oteleReferans(Nokta& n)` imzasındaki `&` işaretini fark edin — bu, `oteleDeger` ile aynı gövdeye sahip görünen bu fonksiyonun aslında çok farklı bir şey yaptığını söyler. `n`, çağıran tarafta verilen gerçek nesneye doğrudan bağlanan bir takma addır; ayrı bir kopya hiç oluşmaz. Gövdedeki `n.x += 10.0;` satırı bu kez çağırandaki nesnenin kendisini değiştirir — fonksiyon geri döndüğünde bu değişiklik kalıcıdır, çünkü `n` hiçbir zaman ayrı bir nesne olmamıştır, en başından beri çağırandaki nesnenin ta kendisiydi.

2.2.3 `const` referansla geçme: kopyasız ve değiştirilemez

Referansla geçme kopyayı önler, ama bir bedeli vardır: fonksiyonu çağıran kişi, gövdenin argümanı değiştirip değiştirmeyeceğini imzaya bakmadan kestiremez — `oteleReferans(n)` çağrısı, tıpkı `oteleDeger(n)` gibi görünür, ama biri `n`'yi değiştirir, diğeri değiştirmez. Çoğu zaman istediğiniz şey ikisinin ortasıdır: kopya maliyetinden kaçınmak (referansın verdiği), ama aynı zamanda fonksiyonun argümanı

değiştiremeyeceğine dair bir güvence de istemek. Hafta 1'de `const std::string&` ile bu kalıbı zaten gördünüz; burada aynı fikri `Nokta` üzerinde pekiştiriyorsunuz:

```
22 double oteleConstReferans(const Nokta& n) {
23    // yalnizca okur; n uzerinde degisiklik yapamaz (const)
24    return n.x + n.y;
25 }
```

`oteleConstReferans(const Nokta& n)` imzasını iki parçaya ayırarak okuyun. `&` işareti kopyayı önler — tıpkı bir önceki alt bölümde gördüğünüz gibi, `n` çağırandaki nesneye doğrudan bağlanır, ayrı bir `Nokta` oluşturulmaz. `const` anahtar kelimesi ise bu bağın *salt okunur* olduğunu derleyiciye söyler: gövde içinde `n.x = ...` gibi bir yazma girişimi olsaydı, derleyici bunu reddederdi. Yukarıdaki gövdenin `n.x + n.y` biçiminde yalnızca *okuma* yapması bu sözle tutarlıdır; fonksiyon çağırandaki nesneye dokunmadan yalnız bir toplam hesaplayıp döndürür.

Bu kalıp — kopyasız *ve* değiştirilemez — büyük bir nesneyi yalnızca okumak istediğinizde neredeyse her zaman doğru tercihtir: bir tarafta değere göre geçmenin kopyalama maliyetinden, öte tarafta referansla geçmenin "fonksiyon bunu değiştirebilir mi?" belirsizliğinden aynı anda kurtulursunuz. Bu yüzden `const T&`, büyük bir tipi salt okumak için bu haftanın — ve genel olarak C++'ın — varsayılan tercihidir.

2.2.4 İşaretçiyle geçme: adres verirsiniz, boş olabilir

C'den zaten tanıdığınız dördüncü biçim işaretçidir. Referansla arasındaki fark, geçen hafta gördüğünüz tanımın doğrudan bir sonucudur: bir referans asla boş olamazken, bir işaretçi boş (`nullptr`) olabilir.

Tanım: İşaretçiyle geçme

İşaretçiyle geçme (*pass by pointer*, `T*`), bir parametrenin çağırandaki nesnenin *adresini* taşıyan bir işaretçi olarak geçirilmesidir. Çağıran taraf `&` operatörüyle adresi açıkça almalıdır; parametre boş (`nullptr`) olabilir — hiçbir nesneyi göstermeyebilir — ve fonksiyon içinde başka bir nesnenin adresine yeniden atanabilir.

Analoji

İşaretçiyle geçmeyi, belgenin kendisini değil, belgenin saklandığı dolabın *numarasını* vermeye benzetebilirsiniz. Numarayı alan kişi önce dolaba gitmek — numarayı "açmak" — zorundadır ve numara boş da olabilir (hiçbir dolabı göstermeyebilir). Böyle bir durumda numarayı kontrol etmeden dolaba gitmeye çalışmak sizi boş bir dolabın önünde bırakır; bu yüzden numarayı kullanmadan önce her zaman "bu numara gerçekten bir dolabı gösteriyor mu" diye kontrol etmeniz gerekir.

```
29 void oteleIsaretci(Nokta* n) {
30    if (n == nullptr) {
31       return;
32    }
33    n->x += 10.0;
34    n->y += 10.0;
35 }
```

`oteleIsaretci(Nokta* n)` imzasında `*` işareti `n`'nin bir `Nokta` işaretçisi olduğunu, yani bir `Nokta` nesnesinin adresini tuttuğunu söyler. Gövdenin ilk satırı — `if (n == nullptr) return;` — bu ünitenin en önemli alışkanlığını gösterir: bir işaretçiyi *kullanmadan önce* her zaman boş olup olmadığını kontrol edin. Bu kontrol yapılmadan doğrudan `n->x` yazmaya çalışsaydınız ve `n` gerçekten `nullptr` olsaydı, program tanımsız biçimde çökerdi. Kontrolden geçen `n`, `n->x += 10.0;` satırında `->` operatörüyle — C'deki gibi — işaret ettiği nesnenin alanına erişir ve onu değiştirir; bu değişiklik, referansla geçmede olduğu gibi, çağırandaki gerçek nesneye yansır, çünkü `n` o nesnenin adresini tutmaktadır.

Referans ile işaretçi burada aynı işi — çağırandaki nesneyi değiştirmeyi — yapıyor gibi görünse de aralarındaki fark önemlidir: bir referans daima geçerli bir nesneye bağlıdır, bu yüzden onu kullanmadan önce boşluk kontrolüne hiç gerek yoktur; bir işaretçi ise boş olabileceği için her kullanımdan önce bu kontrolü *sizin* elle yapmanız gerekir. Bu yüzden işaretçiyi yalnızca parametrenin gerçekten isteğe bağlı/boş olabilir olduğu durumlarda tercih edin — referansın karşılayamadığı, tam olarak bu tek durumdur.

2.2.5 Dördünü yan yana görmek

Aşağıdaki `main` fonksiyonu, aynı başlangıç `Nokta`'sı üzerinde dört fonksiyonu sırayla çağırıp sonuçları karşılaştırır:

```
39 int main() {
40    Nokta baslangic{1.0, 1.0};
41
42    Nokta degerNoktasi = baslangic;
43    oteleDeger(degerNoktasi);
44    std::cout << "deger ile -> otelemeden sonra (degismedi): ";
45    geometri::yazdir(degerNoktasi);
46    std::cout << '\n';
47
48    Nokta referansNoktasi = baslangic;
49    oteleReferans(referansNoktasi);
50    std::cout << "referans ile -> otelemeden sonra (degisti) : ";
51    geometri::yazdir(referansNoktasi);
52    std::cout << '\n';
53
54    Nokta constNoktasi = baslangic;
55    double toplam = oteleConstReferans(constNoktasi);
56    std::cout << "const ref ile -> nokta degismedi, okundu : ";
57    geometri::yazdir(constNoktasi);
58    std::cout << " (x+y=" << toplam << ")\n";
59
60    Nokta isaretciNoktasi = baslangic;
61    oteleIsaretci(&isaretciNoktasi);
62    std::cout << "isaretci ile -> otelemeden sonra (degisti) : ";
63    geometri::yazdir(isaretciNoktasi);
64    std::cout << '\n';
65
66    return 0;
67 }
```

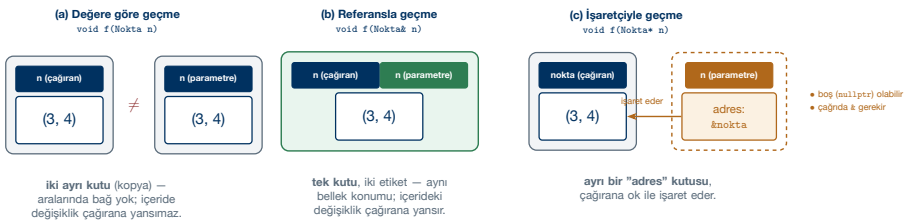
Örnek Olay 2.1: Dört çağrının sonucunu satır satır izlemek

`baslangic` noktası `(1.0, 1.0)` olarak tanımlanır; sonraki dört adımda bu değer her seferinde yeni bir kopyaya (`degerNoktasi`, `referansNoktasi`, `constNoktasi`, `isaretciNoktasi`) atanır, böylece her çağrı aynı başlangıç değeriyle, birbirinden bağımsız başlar.

1. **Değer ile:** `oteleDeger(degerNoktasi)` çağrılır. Fonksiyon içinde bir kopya oluşur ve o kopya ötelenir; ama `degerNoktasi`'nin kendisi hiç dokunulmamış gibi `(1.0, 1.0)` olarak kalır. Ekrandaki satır bunu "degismedi" notuyla doğrular.
2. **Referans ile:** `oteleReferans(referansNoktasi)` çağrılır. Bu kez kopya yoktur; `referansNoktasi`'nin kendisi ötelenir ve `(11.0, 11.0)` olarak çıkar. Ekrandaki satır "degisti" notuyla bunu doğrular.
3. **`const` referans ile:** `oteleConstReferans(constNoktasi)` çağrılır. Fonksiyon yalnızca okur, hiçbir alana yazmaz; `constNoktasi` çağrı sonrası da `(1.0, 1.0)` olarak kalır, yalnızca `x+y` toplamı — `2.0` — geri döner.
4. **İşaretçi ile:** `oteleIsaretci(&isaretciNoktasi)` çağrılır — dikkat edin, çağıran tarafta `&` ile adres açıkça alınır (referansla geçmede bu adımı hiç yazmamıştınız). İçeride `nullptr` kontrolü geçilir, işaret edilen nesne ötelenir ve `isaretciNoktasi` de `(11.0, 11.0)` olarak çıkar.

Sonuç: dört çağrının ikisi (referans ve işaretçi) çağırandaki nesneyi gerçekten değiştirir, ikisi (değer ve `const` referans) değiştirmez — ayrım imzadaki `&`/`const` işaretlerinden okunur, koddaki başka hiçbir yerden değil.

Şekil 2.1 bu üç temel biçimi bellek düzeyinde özetliyor: değere göre geçmede çağıran ve fonksiyon içi iki ayrı kutudur; referansla geçmede tek bir kutuya iki isim iğnelenmiştir; işaretçiyle geçmede ise çağırandaki kutudan ayrı, ona bir okla işaret eden ve boş olabilen bir "adres" kutusu vardır.



Şekil 2.1: Değere göre geçme (iki ayrı kutu/kopya), referansla geçme (tek kutu, iki isim) ve işaretçiyle geçme (ayrı adres kutusu, boş olabilir) arasındaki fark.

2.2.6 Karar tablosu: hangisini ne zaman seçersiniz?

Dört biçimi öğrendikten sonra asıl soru şudur: yeni bir fonksiyon yazarken hangisini seçeceksiniz? Aşağıdaki basit kural kümesi, bu haftaki örneklerin tamamından çıkardığınız fikri özetler.

Durum	Tercih
Küçük tip (`int`, `double`, `Nokta` gibi)	Değere göre (`T`)
Büyük tip, fonksiyon değiştirecek	Referansla (`T&`)
Büyük tip, fonksiyon yalnız okuyacak	`const` referansla (`const T&`)
Parametre isteğe bağlı/boş olabilir	İşaretçiyle (`T*`)

Ortak Hata / Tuzak

Büyük bir tipi (uzun bir `std::string`, yüzlerce elemanlı bir liste gibi) hiç değiştirmeyecekken değere göre geçmek — yani `void yazdir(std::string metin)` yazıp `void yazdir(const std::string& metin)` yazmamak — yaygın, performans açısından sinsi bir alışkanlıktır. Kod *çalışır*, çıktısı doğrudur, bu yüzden hata fark edilmeden kalabilir; ama her çağrıda gereksiz bir kopya oluşturulur. Tipi küçükse (bir `int`, bir `Nokta`) bu maliyet önemsizdir — kuralı katı bir dogma değil, yukarıdaki tabloyu izleyen bir alışkanlık olarak görün.

Bu ünitede gördüğünüz fikir kısaca şudur: bir parametreyi değere göre geçmek kopya oluşturur ve çağırandan bağımsızlaştırır; referansla geçmek kopyasız bir takma ad verir ve değişikliği çağırana yansıtır; `const` referansla geçmek bu ikisinin en iyi yönlerini — kopyasızlık ve değiştirilemezlik — birleştirir; işaretçiyle geçmek ise C'den tanıdık adres tabanlı geçişi, boş olabilme esnekliğiyle birlikte sürdürür. Gelecek ünitede bu değişkenlerin — ve genel olarak her nesnenin — *nerede* yaşadığına, yığın ile öbek arasındaki farka bakacaksınız.

3 • Katılım **Şimdi Siz Deneyin**

`void f(Nokta n)` fonksiyonu içinde `n.x` değiştirilirse çağıranın `Nokta`'sı etkilenir mi? Peki `void f(Nokta& n)` içinde? Büyük bir `Nokta`'yı yalnız *okumak* (toplamını almak) istiyorsanız hangi biçimi seçersiniz ve neden?

Geribildirim. `f(Nokta n)`'de bir kopya oluşur; `n` çağırandaki nesneden bağımsızdır, değişiklik çağırana *yansımaz*. `f(Nokta& n)`'de `n`, çağırandaki nesneye verilmiş ikinci bir isimdir (kopya yok); değişiklik doğrudan çağırana *yansır*. Salt okuma için `const Nokta&` seçilir: kopyasız olduğu için maliyetten kaçınır, `const` olduğu için nesneyi değiştiremeyeceğinizi hem derleyiciye hem okuyucuya garanti eder — büyük bir nesneyi yalnız okumak için en güvenli varsayılan tercihtir.

Kendinizi Sınayın 2.2

`void f(Nokta n)` içinde `n.x` değiştirilirse çağıranın `Nokta`'sı neden etkilenmez, ama `void f(Nokta& n)` içinde etkilenir? Yanıtınızda kopya kavramını kullanın.

5 • İzleme **Aktarım — Başka Nerede?**

Karar tablosu — küçük tip → değere; büyük ve değişecek → `T&`; büyük ve değişmeyecek → `const T&`; opsiyonel/boş olabilir → `T*` — her fonksiyon imzasında yeniden karşınıza çıkar. İleride göreceğiniz Kitaplık projesinde bir `Kitap`'ı listeye eklerken onu gereksiz kopyalamamak için `const Kitap&` ile geçmek, bir

kaydı yerinde güncellemek için `Kitap&` ile geçmek aynı kararın uygulamasıdır. Bulunmayabilecek bir üyeyi temsil eden bir parametrenin `nullptr` olabilmesi ise referansın karşılayamadığı tek durumu — "bu değer hiç olmayabilir" — çözer.

2.3 Bellek Modeli: Yığın, Öbek, `new/delete`, `nullptr`

1 • Ön-öğretim **Neden Önemli?**

Bir fonksiyonun içinde oluşturduğunuz bir nesneyi çağırana döndürmek istediniz; ama fonksiyon bitince o nesne "yok oldu" ya da geri döndürdüğünüz adres çöp bir değere işaret etti. Bir programın uzun süre çalışırken belleğinin durmadan şişip sonunda tükenmesi de aynı ailedendir. Bunların hepsi tek bir soruya bakar: bu bellek "otomatik mi" yoksa "sizin mi" ve onu geri vermeyi kim üstlenir? Bu ünite yığın ile öbeği ayırarak belleğin kimin sorumluluğunda olduğunu netleştirir.

1 • Ön-öğretim **Ön Bilgi**

Bu üniteye girmeden önce C'den `malloc/free`, işaretçi ve `NULL` deneyiminizi (bu haftanın `nullptr`'ı ile karşılaştıracaksınız), Hafta 1'in `geometri::Nokta` değer tipini ve kapsam (`{ }` bloğu) kavramını, Ünite 2'nin işaretçiyle geçme biçimini hatırlayın. Bir değişkenin "ne zaman doğar, ne zaman ölür" sorusu (ömür) bu ünitenin eksenidir.

Önceki ünitede parametrelerin bir fonksiyona nasıl ulaştığına baktınız; şimdi bir adım geri gidip daha temel bir soruyu sorun: bir değişken tanımladığınızda o değişken *nerede* yaşar, ne zaman doğar, ne zaman ölür? C'de `malloc/free` ile dinamik bellek ayırmayı zaten biliyorsunuz; bu ünite o bilgiyi C++'ın kendi sözdizimiyle — `new/delete` — ve C++'a özgü iki kavramla — yığın (stack) ile öbek (heap) arasındaki ayrım, tip güvenli boş işaretçi `nullptr` — yeniden çerçeveler.

2.3.1 Yığın (stack): otomatik ömür

Bir fonksiyonun içinde sıradan bir yerel değişken tanımladığınızda — `Nokta yerel{5.0, 5.0};` gibi — bu değişken bir bellek bölgesinde yaşar ve o bölgenin kuralları tamamen otomatiktir:

Tanım: Yığın (stack)

Yığın (*stack*, çağrı yığını), her fonksiyon çağrısında büyüyüp fonksiyon geri döndüğünde küçülen bir bellek bölgesidir. Bir fonksiyon/blok içinde tanımlanan yerel değişkenler bu bölgede otomatik olarak ayrılır; fonksiyonun/bloğun kapsamı sona erdiğinde aynı bellek otomatik olarak geri alınır — siz hiçbir şey yazmazsınız.

Bu "otomatik" sözcüğünü ciddiye alın; C++'ta bu davranışın kendi adı vardır: **otomatik ömür** taşıyan bir değişken, tanımlandığı kapsamdan çıktığı anda — fonksiyonun kapanış parantezine gelindiğinde — ayrılan bellek otomatik olarak geri kazanılır.

otomatik ömür: bir değişkenin yaşam süresinin, içinde bulunduğu kapsamla (fonksiyon/blok) otomatik sınırlı olması (*automatic storage duration*) — kapsam başladığında bellek ayrılır, kapsam bittiğinde

```
8  void stackOrnegi() {
9     Nokta yerel{5.0, 5.0};
10    std::cout << "stack nesnesi adresi: " << &yerel << '\n';
11 }
```

`stackOrnegi()` çağrıldığında `yerel` adlı `Nokta` nesnesi yığında ayrılır; adresini ekrana yazdırırsınız, ama bu adresi elle saklamaya ya da fonksiyon dışına taşımaya çalışmayın — `stackOrnegi()` geri döndüğü anda `yerel`'in ayrıldığı bellek otomatik geri alınır, o adres artık geçersizdir. Yığın ayırması hızlıdır — yalnızca yığının tepe göstergesini bir miktar ilerletmekten ibarettir — ama boyutu ve ömrü programcının doğrudan kontrolünde değildir: bir değişkenin ne kadar yaşayacağına, kapsamı belirler, siz değil.

2.3.2 Öbek (heap): dinamik ömür, `new/delete`

Bazen bir nesnenin bir fonksiyonun kapsamından *bağımsız* yaşamasını istersiniz — örneğin bir fonksiyonda oluşturduğunuz bir nesneyi başka bir fonksiyona döndürmek, ya da programın çalışması boyunca yaşamasını istediğiniz bir kaynak kurmak. Yığın bunu karşılayamaz — kapsam bitince otomatik silinir. C++ bunun için ikinci bir bellek bölgesi sunar:

> **Tanım: Öbek (heap)**
>
> *Öbek* (*heap*, serbest bellek alanı), `new` anahtar kelimesiyle *açıkça* istenip `delete` ile *açıkça* geri verilen bir bellek bölgesidir. Öbekte ayrılan bir nesne, hiçbir fonksiyonun/bloğun kapsamına bağlı değildir — siz `delete` çağırana kadar yaşamaya devam eder, bir fonksiyondan `new` ile ayrılan bellek başka bir fonksiyona döndürülüp orada kullanılabilir.

dinamik ömür: bir nesnenin yaşam süresinin programcı tarafından `new/delete` ile açıkça belirlenmesi (*dynamic storage duration*) — kapsamdan tamamen bağımsızdır.

Bu kalıcılığın adı **dinamik ömür** taşır — yaşam süresini artık kapsam değil, sizin `new`/`delete` çağrılarınız belirler.

```
15 void newDeleteOrnegi() {
16    Nokta* dinamik = new Nokta{7.0, 7.0};
17    std::cout << "dinamik nesne adresi: " << dinamik << '\n';
18    delete dinamik;
19    dinamik = nullptr;
20 }
```

`new Nokta{7.0, 7.0}` ifadesini iki adımda okuyun: önce öbekte bir `Nokta` boyutunda yer ayrılır, sonra bu yer `{7.0, 7.0}` değerleriyle doldurulur (bu doldurma adımına ileride, Hafta 3'te, *kurucu* adını vereceksiniz — burada yalnızca "`new` hem yer ayırır hem nesneyi kurar" gözlemi yeterlidir). İfadenin sonucu bu yerin *adresidir*; bu yüzden `dinamik` bir `Nokta*` işaretçisine atanır. `delete dinamik;` satırı ise tersini yapar: önce nesnenin kapanış adımı çalışır (adına *yıkıcı* diyeceksiniz — Hafta 3 Ünite 2'nin konusu), sonra bellek öbeğe geri verilir. `delete`'ten hemen sonra işaretçiyi `nullptr`'a çekmek — `dinamik = nullptr;` — iyi bir alışkanlıktır; nedenini birazdan sarkan işaretçi bölümünde göreceksiniz.

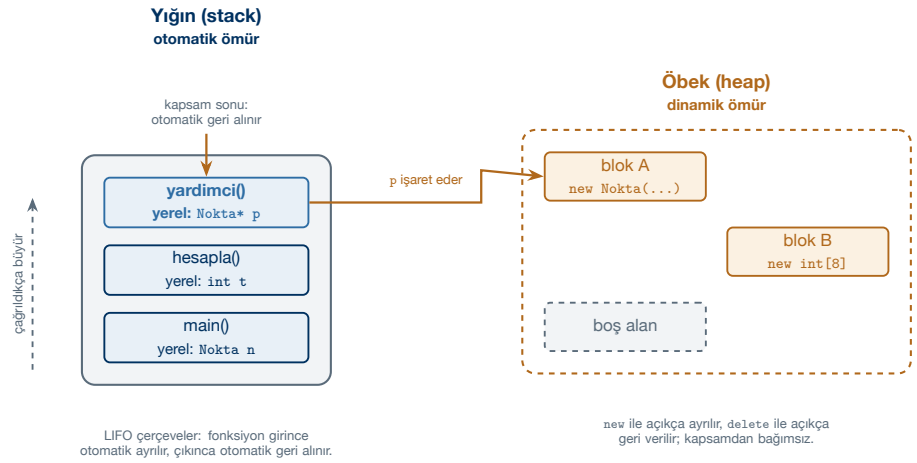
Bu sözdizimini C'nin `malloc`/`free`'siyle karşılaştırın: `malloc(sizeof(Nokta))` boyutu elle hesaplamanızı ve dönen `void*`'ı elle doğru tipe çevirmenizi (*cast*) ister — derleyici burada size hiçbir tip denetimi sağlamaz, yanlış boyut ya da yanlış çevirim sessizce "çalışır", hata çalışma zamanında ortaya çıkar. `new Nokta{...}` ise boyutu kendisi hesaplar, sonucu zaten doğru tipte (`Nokta*`) döndürür — Ünite 1'in ve Hafta 1'in sürekli vurguladığı tip güvenliği ilkesinin burada da geçerli olduğunu

görürsünüz. `new` bir dizi biçimini de destekler — `new T[n]` / `delete[] p` — bunu Ünite 4'te `IntTampon` örneğinde somut olarak göreceksiniz.

Analoji

Yığın ile öbek arasındaki farkı masaüstünüz ile kiraladığınız bir depoya benzetebilirsiniz. Masaüstünüzdeki kağıtlar (yığın/otomatik değişkenler) işiniz bitince kendiliğinden toplanır — odayı terk ettiğinizde (kapsam sona erdiğinde) kimse size "şu kağıtları topla" demek zorunda kalmaz, bu otomatik olur. Depo (öbek) ise farklıdır: siz gidip kiralamadıkça (`new`) hiçbir depo size ait olmaz, ve siz anahtarı iade etmedikçe (`delete`) o depo boşalmaz — depo, sizin ofis kapınızı kapatıp gitmenizden (fonksiyonun bitmesinden) tamamen bağımsız olarak sizin adınıza kayıtlı kalır.

Şekil 2.2 bu iki bölgeyi yan yana özetliyor: solda yığın — fonksiyon çağrıları büyüdükçe/küçüldükçe otomatik ayrılıp geri alınan çerçeveler; sağda öbek — `new` ile açıkça ayrılan, `delete` ile açıkça geri verilen, kapsamdan bağımsız dağınık bloklar; yığındaki bir işaretçi değişkeninden öbekteki bir bloğa giden bir ok.



Şekil 2.2: Yığın (stack): kapsamla otomatik ayrılıp geri alınan çerçeveler. Öbek (heap): `new`/`delete` ile açıkça yönetilen, kapsamdan bağımsız bloklar.

Karmaşıklık (big-O)

Yığında bir değişken ayırmak sabit zamanlıdır: derleyici, fonksiyon çağrıldığı anda o fonksiyonun tüm yerel değişkenleri için gereken toplam alanı zaten bilir, yığının tepe göstergesini bu miktar kadar ilerletmek tek işlemdir — $O(1)$, hem de son derece küçük bir sabitle. Öbekte bir nesne ayırmak (`new`) böyle değildir: bellek ayırıcı (*allocator*), önceki `delete`'lerden kalan boş blokları takip eden bir defter tutar ve istenen boyuta uygun boş bir blok bulmak zorundadır — bu arama, öbeğin o anki durumuna (parçalanmışlığına) bağlı olarak değişken bir maliyet taşır. Bu yüzden sık ve küçük nesneler için tekrar tekrar `new`/`delete` çağırmak, aynı işi yığında yapmaktan gözle görülür biçimde daha pahalıdır.

2.3.3 `nullptr`: tip güvenli boş işaretçi

C'de bir işaretçinin "hiçbir yeri göstermediğini" `NULL` makrosu ya da doğrudan `0` ile işaretlersiniz; ikisi de arka planda tamamen sıradan bir tam sayı sabitidir, derleyici bir işaretçi bağlamında mı yoksa bir sayı bağlamında mı kullanıldığınızı her zaman güvenle ayırt edemez. C++ bunun yerine **nullptr** sunar.

nullptr: hiçbir nesneyi göstermeyen, tip-güvenli boş işaretçi değeri (*null pointer literal*); C'nin `NULL`/`0` makrosundan farklı olarak yalnızca işaretçi bağlamında geçerlidir, bir tam sayıyla asla karışmaz.

```
void nullptrKontrolOrnegi() {
   Nokta* p = nullptr;
   if (p == nullptr) {
      std::cout << "isaretci bos (nullptr), erisim yapilmiyor\n";
   } else {
      std::cout << "isaretci dolu: " << p->x << '\n';
   }
}
```

`Nokta* p = nullptr;` satırı `p`'yi açıkça "hiçbir nesneyi göstermiyor" durumuna getirir. Bir sonraki satırdaki `if (p == nullptr)` kontrolü, Ünite 2'de işaretçiyle geçmede gördüğünüz alışkanlığın aynısıdır: bir işaretçiyi `->` ya da `*` ile kullanmadan *önce*, gerçekten geçerli bir nesneyi gösterip göstermediğini kontrol edin. Bu program `p` boşken `p->x` satırına hiç girmez — kontrol olmasaydı bu satır tanımsız davranışa yol açardı.

> **İpucu**
>
> Bir işaretçi tanımlarken, ona hemen bir değer veremiyorsanız bile onu `nullptr` ile ilkleyin — `Nokta* p;` yerine `Nokta* p = nullptr;` yazın. İlklenmeyen bir yerel işaretçi rastgele/çöp bir adres taşır; bu adres "boş görünmeyebilir" bile, yani `if (p == nullptr)` kontrolü onu yakalamaz, ama işaret ettiği yer aslında geçerli değildir. `nullptr` ile ilklemek, en azından kontrolünüzün gerçekten işe yaramasını garanti eder.

2.3.4 Bellek sızıntısı: unutulan `delete`

`new` ile öbekte ayrılan her bellek, karşılığı bir `delete` çağrılana kadar sizin sorumluluğunuzdadır. Bu sorumluluk atlanırsa ortaya çıkan, yani `new` ile ayrılmış ama karşılığı hiç `delete` edilmeyen belleğin erişilemez biçimde "kayıp" kalması durumuna **bellek sızıntısı** denir.

bellek sızıntısı: `new` ile ayrılan belleğin karşılığı `delete` edilmeden erişilemez hale gelmesi (*memory leak*) — bellek program sonlanana kadar geri alınamaz.

```
// DIKKAT: bu fonksiyon KASITLI olarak hatalidir; main() tarafindan CAGRILMAZ
    .
// new ile ayrilan bellegin karsiligi hicbir zaman delete edilmiyor.
void sizintiOrnegi() {
   Nokta* p = new Nokta{1.0, 1.0};
   p = new Nokta{2.0, 2.0}; // ilk ayrilan adres burada kaybolur, artik
       delete edilemez
   // delete p; // <- kasitli olarak cagrilmadi: bellek sizintisi bu satirin
       eksikliginden dogar
   (void)p;
}
```

Bu fonksiyonu adım adım izleyin. İlk satırda `p`, öbekte ayrılan ilk `Nokta`'nın adresini tutar. İkinci satırda `p`'ye *ikinci* bir `new` çağrısının sonucu atanır — bu satır çalıştığında `p` artık ikinci nesnenin adresini tutar, birinci nesnenin adresi ise hiçbir değişkende kalmaz; o adresi bir daha asla elde edemezsiniz, dolayısıyla ona karşılık gelen belleği asla `delete` edemezsiniz. Birinci nesne, program sonlanana kadar öbekte "kilitli" kalır — artık kimse ona erişemez, ama işletim sistemi onu geri

alamaz, çünkü siz `delete`'i hiç çağırmadınız. Bu fonksiyon bilerek `main` tarafından çağrılmaz; yalnızca deseni göstermek için kaynak dosyada durur, böylece kodun tamamı güvenle derlenip çalışabilir.

Analoji

Bellek sızıntısını, kiraladığınız bir depoyu kullanmayı bıraktığınız hâlde anahtarını hiç iade etmemenize benzetebilirsiniz. Depo artık kimsenin — sizin de dahil — kullanamayacağı biçimde "kayıp" durur; siz onu tamamen unutsanız bile kira hesabınız (programınızın kullandığı bellek miktarı) işlemeye devam eder. Programınız uzun süre çalışırsa, art arda unutulan her "kiralık depo" toplam bellek kullanımını sessizce büyütür — ta ki bellek tükenip program çökene kadar.

2.3.5 Sarkan işaretçi: geçersiz belleğe bakan işaretçi

Sızıntının tam tersi bir tuzak da vardır: belleği doğru şekilde `delete` etmek, ama o adresi gösteren işaretçiyi kullanmaya devam etmek. `delete`'ten sonra hâlâ o eski adresi taşımaya devam eden, ama artık geçerli bir nesne göstermeyen bu işaretçiye **sarkan işaretçi** denir.

sarkan işaretçi: Bir zamanlar geçerli bir nesneyi gösterirken, o nesnenin belleği `delete` ile geri verildiği (ya da kapsam sonunda yok olduğu) için artık geçersiz bir adresi tutan işaretçi (*dangling pointer*). İşaretçinin değeri değişmez; ama gösterdiği bellek artık ona ait olmadığından üzerinden erişim tanımsız davranıştır (*undefined behavior*).

```
// DIKKAT: bu fonksiyon KASITLI olarak sarkan isaretci DESENINI gosterir;
// main() tarafindan CAGRILMAZ ve p uzerinden gercek bir erisim (dereference)
     YAPILMAZ.
void sarkanOrnegi() {
   Nokta* p = new Nokta{3.0, 3.0};
   delete p;
   // p artik sarkan (dangling) bir isaretci: delete edilmis bellegi
       gosteriyor.
   // Iyi aliskanlik: delete sonrasi "p = nullptr;" yapmaktir (burada bilerek
        atlandi).
   // p->x = 9.0; // <- YAPILMAMALIDIR: tanimsiz davranis (UB); bu satir
       hicbir zaman calismaz.
}
```

`delete p;` çalıştıktan sonra `p` hâlâ eski adresi taşımaya devam eder — `delete` işaretçinin *değerini* değiştirmez, yalnızca o adresteki belleği öbeğe geri verir. Bu andan itibaren `p` bir sarkan işaretçidir: artık "sahibi olmadığı" bir yeri gösterir, o bellek başka bir `new` çağrısı tarafından yeniden kullanılmış bile olabilir. Yorum satırındaki `p->x = 9.0;` satırı kasıtlı olarak *çalıştırılmaz* — kodda yalnızca deseni göstermek için durur; gerçekten çalıştırılsaydı davranışı tanımsız olurdu, bu bazen sessizce "çalışıyormuş gibi" görünebilir, bazen de programı çökertebilir.

Analoji

Sarkan işaretçiyi elinizdeki eski bir depo anahtarına benzetebilirsiniz. Deponun sözleşmesi çoktan bitmiştir (bellek `delete` edilmiştir), belki depo şimdiden başka birine kiralanmıştır (bellek başka bir `new` tarafından yeniden kullanılmıştır); o eski anahtarla depoya girmeye çalışmak tehlikelidir — içeride ne olduğu artık sizin kontrolünüzde değildir, belki boş bir oda, belki başka birinin eşyaları vardır.

Dikkat

Bu ünitedeki `sizintiOrnegi()` ve `sarkanOrnegi()` fonksiyonları kaynak dosyada bilerek durur ama `main` tarafından *çağrılmaz*; sarkan işaretçi örneğinde de işaretçi üzerinden gerçek bir erişim (*dereference*) hiçbir zaman çalıştırılmaz. Bu sayede kod tabanının tamamı güvenle derlenip çalışır — amaç size tanımsız davranışı gerçekten tetiklettirmek değil, deseni tanımanızı sağlamaktır.

Ortak Hata / Tuzak

İyi bir alışkanlık, `delete p;` çağrısından hemen sonra `p = nullptr;` yazmaktır. Bu, sarkan işaretçi sorununu tamamen ortadan kaldırmaz — `p`'nin bir kopyası başka bir değişkende duruyorsa o kopya hâlâ sarkan kalır — ama en azından `p`'nin kendisi üzerinden yapılacak sonraki bir yanlış kullanım artık sessizce tanımsız davranışa değil, `if (p == nullptr)` ile kolayca yakalanabilecek bir duruma dönüşür. Bu ünitenin `sarkanOrnegi()` fonksiyonu bu adımı bilerek atlar, tam da bu eksikliği size göstermek için.

Bu ünitede gördüğünüz fikir kısaca şudur: yığında yaşayan değişkenlerin ömrünü kapsam belirler ve bu tamamen otomatiktir; öbekte yaşayan nesnelerin ömrünü ise siz `new`/`delete` ile açıkça belirlersiniz — bu özgürlük iki yeni sorumluluk getirir: her `new`'in bir `delete`'i olmalı (yoksa sızıntı), ve `delete` edilmiş bir işaretçi bir daha kullanılmamalı (yoksa sarkan işaretçi). Gelecek ünitede bu iki riski — unutma ve yanlış kullanım — elle yönetmek yerine bir nesnenin ömrüne bağlayarak nasıl azaltabileceğinizi, RAII fikrinin ilk somut örneğiyle göreceksiniz.

3 • Katılım — Şimdi Siz Deneyin

Şu kod bir bellek sızıntısı oluşturur mu, oluşturursa nerede: önce `Nokta* p = new Nokta{1,1};`, ardından `p = new Nokta{2,2};`, ardından `delete p;`? Ayrıca `nullptr` olan bir işaretçiyi `p->x` ile okumaya çalışırsanız ne olur?

Geribildirim. Evet, sızıntı oluşur. İlk `new` ile ayrılan nesnenin adresi ikinci atamayla kaybolur — `p` artık ikinci nesneyi gösterir. `delete p;` yalnız *ikinci* nesneyi bırakır; ilk nesnenin karşılığı olan `delete` hiç çağrılmaz, o bellek program bitene kadar geri alınamaz. Boş (`nullptr`) bir işaretçiyi `p->x` ile okumak ise hiçbir nesneyi göstermeyen bir adrese erişmektir; tanımsız davranıştır (tipik olarak çökme). Bu yüzden bir işaretçiyi kullanmadan önce `nullptr` kontrolünü alışkanlık hâline getirirsiniz.

Kendinizi Sınayın 2.3

`new` ile ayrılan bir nesne için karşılık gelen `delete` hiç çağrılmazsa ne olur? Yanıtınızda bellek sızıntısı kavramını kullanın.

5 • İzleme — Aktarım — Başka Nerede?

"Her `new` bir `delete` ister; otomatik olan yığın, sizin olan öbek" ayrımı, dinamik bellek kullanan her yapıda geçerlidir. İleride göreceğiniz Kitaplık projesinde çalışma anında büyüyen bir kitap listesi tutan bir yapı, bu belleği öbekten ister; onu geri vermeyi ihmal etmek tam da bu ünitenin sızıntısıdır. Bu gözlem, bir

sonraki ünitenin "geri vermeyi elle hatırlamak yerine nesnenin ömrüne bağlamak" fikrine doğrudan köprü kurar.

2.4 RAII'ye İlk Bakış: Kaynağı Nesne Ömrüne Bağlamak

1 • Ön-öğretim — Neden Önemli?

Ünite 3'te her `new` için elle bir `delete` yazmanız gerektiğini gördünüz. Ama gerçek kodda bir fonksiyonun ortasından erken `return` etmek, atlanan bir kod yolu ya da yalnızca unutmak — tek başına bir sızıntıya yeter. "Her açtığını kapatmayı hatırlamak" ölçek büyüdükçe sürdürülemez bir yüktür. Bu ünite, temizliği hatırlamaya bırakmak yerine nesnenin ömrüne bağlayan yolu ilk kez somut bir örnekle gösterir.

1 • Ön-öğretim — Ön Bilgi

Bu üniteye girmeden önce Ünite 3'ün `new`/`delete`, sızıntı ve kapsam (blok, otomatik ömür) kavramlarını, ayrıca Hafta 1'de yalnız isim düzeyinde anılan "kapsam sonunda otomatik temizlik" değinisini hatırlayın. Bir yığın nesnesinin kapsam sonunda kendiliğinden yok olduğu (Ünite 3) gözlemi, bu ünitenin fikrinin temelidir: temizliği bu otomatik olaya bağlamak.

Önceki ünitede iki riski ayrıntılı gördünüz: bir `new` çağrısına karşılık gelen `delete`'i unutmak (bellek sızıntısı) ve `delete` edilmiş bir işaretçiyi kullanmaya devam etmek (sarkan işaretçi). İkisinin ortak kökü aynıdır — öbekteki bir kaynağın yönetimi tamamen *size*, programcıya bırakılmıştır; derleyici sizi hiçbir noktada zorlamaz, hatırlamak ya da unutmak tamamen sizin elinizdedir. Bu ünitede bu riski azaltan bir tasarım fikrine ilk kez somut bir örnek üzerinden bakıyorsunuz: Hafta 1'de yalnızca adını duyduğunuz RAII.

2.4.1 Elle yönetimin gerçek riski

Soruna soyut kalmadan, tek bir küçük dizi örneği üzerinden bakın:

```
7    std::cout << "-- elle yonetim --\n";
8    int* elleTampon = new int[5];
9    for (int i = 0; i < 5; ++i) {
10       elleTampon[i] = i * i;
11   }
12   std::cout << "elle tampon dolduruldu\n";
13   // Not: burada delete[] cagrisi UNUTULURSA bellek sizintisi olusur.
14   delete[] elleTampon;
15   std::cout << "elle tampon serbest birakildi\n";
```

Bu kod parçası kusursuz görünür — `new int[5]` ile beş elemanlık bir dizi ayrılır, doldurulur, sonunda `delete[] elleTampon;` ile geri verilir. Sorun şudur: bu doğruluk tamamen *sizin* disiplininize bağlıdır, dilin kendisine değil. Şu değişikliği düşünün: doldurma döngüsünün ortasına bir hata kontrolü ekleyip, bir koşul sağlanırsa fonksiyondan *erken* `return` ettiğinizi varsayın. `delete[] elleTampon;` satırı fonksiyonun *sonunda* durduğu için, erken dönen bu yeni yol o satıra hiç uğramaz — dizi sessizce sızar. Aynı risk bir istisna (*exception*) fırlatılırsa, ya da kod büyüyüp

`delete[]` çağrısı bir `if`/`else` dallanmasının yalnızca bir kolunda kalırsa da geçerlidir: **tek bir atlanan kod yolu, tek bir unutulan satır, sızıntı için yeterlidir** — kaç satırınız doğru olursa olsun, bir tanesi eksikse sonuç aynıdır.

Dikkat

Bu risk yalnızca "dikkatsiz programcı" sorunu değildir. Küçük bir fonksiyonda `new`/`delete` çiftini gözle takip etmek kolaydır; ama gerçek bir kod tabanında bir kaynağın ayrıldığı yer ile geri verildiği yer arasında yüzlerce satır, birden çok erken `return`, hatta birden çok fonksiyon olabilir. Kaynağı elle yönetmek, kod büyüdükçe orantısız biçimde kırılgan hâle gelir.

2.4.2 RAII fikri: kaynağı bir nesnenin ömrüne bağlamak

Hafta 1'de bir cümlelik bir tanıtımla adını duyduğunuz fikri şimdi somutlaştırın:

Tanım: RAII

RAII (*Resource Acquisition Is Initialization*), bir kaynağın (bellek, dosya gibi) edinim ve bırakım işlemlerini bir nesnenin ömrüne bağlayan tasarım ilkesidir: kaynak, nesne **oluşturulduğunda** edinilir; kaynağın bırakılması ise nesnenin **kapsam dışına çıktığı anda otomatik çalışan özel bir fonksiyona** bağlanır. Sonuç olarak kaynağı elle bırakmayı *hatırlamanız* gerekmez — nesnenin kendi ömrü bu işi sizin adınıza garanti eder.

Analoji

RAII'yi bir otel oda kartına benzetebilirsiniz. Kartı check-in'de alırsınız — oda (kaynak) o andan itibaren sizindir; bu, bir nesnenin *oluşturulmasının* kaynağın edinimine karşılık geldiği adımdır. Check-out'ta kartı elden vermeyi unutsanız bile otel, kalış süreniz dolduğunda kapıyı kendiliğinden kilitleyip odayı boşaltır — bu da nesnenin kapsam sonunun kaynağın otomatik bırakılmasına karşılık geldiği adımdır. Dikkat edin: anahtarı "elle iade etmeyi hatırlamanıza" hiç gerek kalmaz; otelin kendi sistemi — tıpkı derleyicinin kapsam sonunda çalıştırdığı özel fonksiyon gibi — bu işi sizin adınıza, unutma ihtimaline hiç yer bırakmadan yürütür.

2.4.3 Somut örnek: `araclar::IntTampon`

Bu fikri elle yazılmış küçük bir sarmalayıcı sınıfla somutlaştırın. `IntTampon`, öbekte bir tam sayı dizisi tutan, ama dizinin bırakılmasını kendi kendine halleden bir tiptir. Önce yapısına bakın:

```
8   int* veri;
9   int boyut;
```

İki alan tanıdıktır: `veri` dizinin öbekteki adresini tutan bir işaretçi, `boyut` ise dizinin kaç elemanlı olduğunu tutan bir `int`'tir. Asıl fark bunun yanında duran iki bildirimde saklıdır:

```
12  IntTampon(int n);
13  ~IntTampon();
```

`IntTampon(int n)` ve `~IntTampon()` adlı bu iki özel üye fonksiyona C++'ta sırasıyla *kurucu* ve *yıkıcı* adını vereceksiniz — bu adları ve biçimsel yazım kurallarını

(üye başlatma listesi, `this` gibi) Hafta 3'te resmî olarak tanımlayacaksınız. Bu ünitede sizden yalnızca dıştan gözlemlenebilir davranışlarını tanımanız istenir: biri — `IntTampon(int n)` — bir `IntTampon` nesnesi **oluşturulduğu anda** derleyici tarafından otomatik çağrılır; diğeri — `~IntTampon()` — nesne **kapsam dışına çıktığı anda** yine derleyici tarafından otomatik çağrılır. Bu iki gövdeye bakın:

```
8  IntTampon::IntTampon(int n) {
9     veri = new int[n];
10    boyut = n;
11    std::cout << "tampon olusturuldu (boyut=" << boyut << ")\n";
12 }
```

```
16 IntTampon::~IntTampon() {
17    delete[] veri;
18    std::cout << "tampon temizlendi\n";
19 }
```

İlk gövde tanıdık geliyor olmalı — `new int[n]` ile öbekte bir dizi ayırır, adresini `veri`'ye yazar, boyutu kaydeder; tam olarak bir önceki alt bölümdeki elle yönetimin ilk adımı. İkinci gövde ise o elle yönetimin siz hiç yazmadığınız hâlde *otomatik* çalışan kapanış adımıdır: `delete[] veri;` ile diziyi öbeğe geri verir. Kritik fark şudur: elle yönetimde `delete[]` çağrısını doğru yere, doğru sırayla, *siz* yazmak zorundaydınız; burada bu çağrı `IntTampon` nesnesinin kendi tanımının içine, bir daha unutulamayacak biçimde gömülüdür.

2.4.4 Elle yönetim ile RAII'yi yan yana görmek

Aynı `main` fonksiyonu içinde iki yaklaşımı art arda görün:

```
19    std::cout << "-- RAII demo --\n";
20    {
21       araclar::IntTampon tampon(5);
22       for (int i = 0; i < tampon.boyut; ++i) {
23          tampon.veri[i] = i * i;
24       }
25       std::cout << "tampon kullaniliyor\n";
26    } // blok sonu: yikici otomatik calisir, tampon kendini temizler
27    std::cout << "blok disina cikildi\n";
```

Örnek Olay 2.2: Bir bloğun içinde `IntTampon` yaşam döngüsü

Yukarıdaki `{ }` bloğunu adım adım izleyin.

1. **Blok başı, oluşturma:** `araclar::IntTampon tampon(5);` satırı çalıştığında derleyici, `IntTampon` nesnesi oluşturulurken otomatik çağrılan özel fonksiyonu (kurucuyu) çalıştırır; konsola "tampon olusturuldu (boyut=5)" yazılır. Bu satır tamamlandığında `tampon` kullanıma hazırdır — öbekte beş elemanlık dizi zaten ayrılmıştır, `tampon.veri` bu diziyi göstermektedir.

2. **Kullanma:** döngü `tampon.veri[i]`'yi doldurur, ardından "tampon kullaniliyor" yazdırılır. Bu adımlar sıradan bir nesne kullanımıdır — `IntTampon` bu noktada elle yönetilen diziden dıştan hiç farklı görünmez.

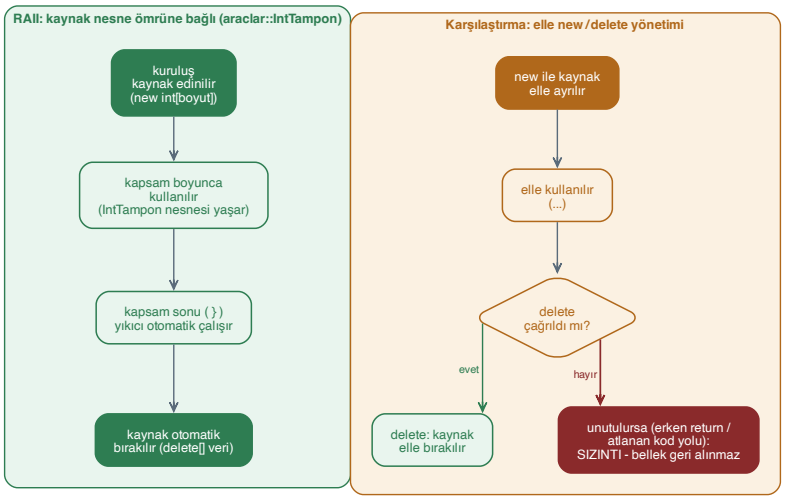
3. **Blok sonu, otomatik temizlik:** kapanış parantezine gelindiğinde — siz hiçbir `delete[]` yazmadan — derleyici `tampon` için kapsam sonunda otomatik çalışan özel fonksiyonu (yıkıcıyı) çağırır; konsola "tampon temiz-

lendi" yazılır. Bu tam olarak Ünite 3'te öğrendiğiniz "kapsam dışına çıkan yerel değişken" kuralının — burada bir `Nokta` yerine bir `IntTampon` için — işlemesidir.

4. **Blok dışı:** program "blok disina cikildi" satırını yazdırır. Bu noktada `tampon` artık yoktur, ama diziyi elle bırakmayı hiç yazmadınız — bunu `IntTampon`'un kapsam sonunda otomatik çalışan kendi özel fonksiyonu (yıkıcısı) sizin adınıza yaptı.

Konsol çıktısını uçtan uca sırayla okursanız iki bölüm görürsünüz: önce elle yönetimin adımları ("elle tampon dolduruldu", "elle tampon serbest birakildi"), ardından RAII bloğunun adımları ("tampon olusturuldu" → "tampon kullaniliyor" → "tampon temizlendi" → "blok disina cikildi"); ikisi de aynı işi — bir tam sayı dizisini ayırıp kullanıp bırakmayı — yapar, ama yalnızca ikincisinde bırakma adımı sizin bir satır daha yazmanızı gerektirmez.

Şekil 2.3 bu fikri genel biçimde özetliyor: bir nesnenin kapsamını temsil eden dikey bir çerçeve, üstte "kuruluş: kaynak edinilir" etiketli bir giriş oku, altta "kapsam sonu: kaynak otomatik bırakılır" etiketli bir çıkış oku taşır; yanında, karşılaştırma için elle yönetilen yol — `new` → ... → `delete` unutulursa kırmızı "sızıntı" çıkışıyla biten küçük bir dal olarak gösterilir.



Şekil 2.3: RAII: kaynağın edinimi nesnenin kuruluşuna, bırakılması kapsam sonuna bağlanır. Elle yönetimde `delete` unutulursa yol sızıntıyla biter.

Ortak Hata / Tuzak

`IntTampon`'u gördükten sonra "madem kapsam dışına çıkınca otomatik çalışan fonksiyon (yıkıcı) var, elle yönetimi hiç öğrenmeme gerek yoktu" diye düşünmek yaygın ama yanlış bir sonuçtur. RAII sarmalayıcısının — yani `IntTampon`'un — *kendi içinde* hâlâ elle yazılmış bir `new`/`delete` çifti vardır; RAII bu çifti *ortadan kaldırmaz*, yalnızca onu tek bir yere, bir daha unutulamayacak biçimde — sınıfın oluşturma/temizlik (kurucu/yıkıcı) çiftine — taşır. Ünite 3'ün bilgisi (ne zaman `new`, ne zaman `delete`, neden sızıntı/sarkan

işaretçi olur) hâlâ tam olarak geçerlidir; RAII bu bilgiyi ortadan kaldırmaz, onu doğru uygulayan kodu bir kez yazıp güvenle tekrar tekrar kullanmanızı sağlar.

Bu ünitede gördüğünüz fikir kısaca şudur: bir kaynağı elle `new`/`delete` ile yönetmek, tek bir atlanan kod yolunda bile sızıntıya yol açabilecek kadar kırılgandır; RAII bu riski, kaynağın bırakılmasını bir nesnenin kapsam sonunda otomatik çalışan özel bir fonksiyona bağlayarak azaltır — `IntTampon` bunun ilk somut örneğiydi. Bu haftanın elle sarmalanmış örneği, gelecek hafta `kurucu`/`yıkıcı`'nın biçimsel kurallarıyla (üye başlatma listesi, `this`) yeniden ele alınacak; Hafta 4'te ise aynı fikir `unique_ptr`/`shared_ptr` gibi hazır akıllı işaretçilerle genelleştirilecek — artık kendi `IntTampon`'unuzu elle yazmanıza bile gerek kalmayacak.

3 • Katılım ⌨ Şimdi Siz Deneyin

Bir diziyi elle `new int[n]` ile ayırıp `delete[]` ile bırakıyorsunuz; fonksiyonun ortasında bir koşul sağlanınca erken `return` ediyorsunuz. Bu neden risklidir? `araclar::IntTampon` gibi, ayırmayı kuruluşunda yapıp bırakmayı kapsam sonunda otomatik çalışan mekanizmasına bırakan bir sarmalayıcı bu riski nasıl ortadan kaldırır?

Geribildirim. Erken `return`, `delete[]` satırına hiç ulaşılmadan fonksiyondan çıkar; dizi bırakılmadan işaretçi kaybolur — sızıntı. `IntTampon`'da bellek nesnenin kuruluşunda ayrılır, bırakılması ise nesnenin kapsam sonunda — blok bitince, hatta erken `return` yolunda bile — otomatik çalışan özel fonksiyonuna bağlıdır; bu yüzden `delete[]`'i elle hatırlamak zorunda kalmazsınız — hangi yoldan çıkarsanız çıkın temizlik kendiliğinden gerçekleşir.

Kendinizi Sınayın 2.4

Bir kaynağı elle `new`/`delete` ile yönetmenin en büyük riski nedir? RAII (bu ünitedeki ilk bakış düzeyinde) bu riski nasıl azaltır?

5 • İzleme ↗ Aktarım — Başka Nerede?

"Kaynağı nesnenin ömrüne bağlama" fikri yalnız belleğe özgü değildir: açık bir dosya, bir ağ bağlantısı veya bir kilit gibi "alınıp geri verilmesi gereken" her kaynak aynı desenle yönetilebilir. İleride göreceğiniz Kitaplık projesinde bir kayıt dosyasını açıp kapatan bir yapı da bu haftanın `IntTampon`'uyla aynı iskeleti kullanır. Bu elle sarmalanmış örnek, Hafta 3'te kurucu/yıkıcının biçimsel kurallarıyla, Hafta 4'te ise `unique_ptr`/`shared_ptr` ile genelleştirilecek fikrin ilk somut adımıdır.

Bölüm Özeti

Özet

Bu hafta fonksiyonları ve C++'ın bellek modelini derinleştirdiniz. İlk ünitede C'de karşılığı olmayan iki araçla tanıştınız: aşırı yükleme, aynı adı taşıyan fonksiyonları imzalarına (ad + parametre listesi, dönüş tipi hariç) göre ayırt etmenizi sağlar — derleyici doğru sürümü çağrı anındaki argümanlara bakarak seçer; varsayılan parametre ise tek bir fonksiyonun bir parametresini isteğe bağlı kılar. İkinci ünitede Hafta 1'in referans girişini `geometri::Nokta` üzerinde dört biçime — değere, referansla, `const` referansla, işaretçiyle geçme — genişlettiniz; her birinin kopya oluşturup oluşturmadığını ve çağırandaki nesneyi değiştirip değiştiremeyeceğini gördünüz, sonunda küçük tip → değere, büyük tip değişecek → `T&`, büyük tip değişmeyecek → `const T&`, opsiyonel/boş olabilir → `T*` kuralına ulaştınız. Üçüncü ünite bir değişkenin *nerede* yaşadığı sorusuna baktı: yığın (stack) otomatik ömürlüdür, kapsam bitince kendiliğinden geri alınır; öbek (heap) dinamik ömürlüdür, `new` ile açıkça istenir, `delete` çağrılana kadar yaşar — bu özgürlük iki riski beraberinde getirir: bellek sızıntısı (unutulan `delete`) ve sarkan işaretçi (`delete` sonrası hâlâ kullanılan işaretçi). Dördüncü ünite bu iki riski azaltan bir fikre — Hafta 1'de yalnızca adını duyduğunuz RAII'ye — ilk kez somut bir örnek üzerinden baktı: `araclar::IntTampon`, kaynağın edinimini bir nesnenin kuruluşuna, bırakılmasını ise kapsam sonunda otomatik çalışan özel bir fonksiyona bağlayarak, elle `new`/`delete` yönetiminin "unutma" riskini ortadan kaldırdı — kaynağı yöneten kod hâlâ oradadır, ama artık tek bir yerde durur ve bir daha atlanamaz. Gelecek hafta fonksiyonları ve belleği bir sınıfın çatısı altında birleştirmeyi — kapsülleme, kurucu/yıkıcı, `this` — göreceksiniz; bu haftanın `IntTampon` içindeki oluşturma/temizlik çifti — Hafta 3'te resmî adlarını (kurucu/yıkıcı) alacak — o haftanın Ünite 2'sinde biçimsel terimlerle yeniden ele alınacak zeminin ta kendisidir.

Kendinizi Sınayın Cevapları

Soru (Ünite 1 — yalnızca dönüş tipiyle ayrılan fonksiyonlar neden derlenmez). İki fonksiyon yalnızca dönüş tipiyle birbirinden ayrılıyorsa (parametre listeleri tıpatıp aynıysa) derlenmez, çünkü bir fonksiyonun *imzası* — derleyicinin iki fonksiyonu birbirinden ayırt etmek için baktığı şey — yalnızca adından ve parametre listesinden oluşur, dönüş tipini hiç içermez. Derleyici bir çağrıyı — örneğin `alan(4)` — yalnızca argüman listesine (kaç argüman, hangi tipte) bakarak hangi sürümle eşleştireceğine karar verir; sonucun nereye atanacağını, dolayısıyla dönüş tipini, çoğu zaman çağrı anında hiç bilmesi gerekmez (`alan(4)`'ün sonucunu bir değişkene atamadan da kullanabilirsiniz). İki `alan(int)` sürümü yalnızca dönüş tipiyle ayrılsaydı, derleyici `alan(4)` çağrısında hangisinin çalışacağına karar veremezdi; bu belirsizliği önlemek için C++ böyle bir tanımı daha derleme aşamasında reddeder.

Soru (Ünite 2 — `Nokta n` ile `Nokta& n` arasındaki fark). `void f(Nokta n)` imzasında `&` işareti yoktur — bu, `n`'nin çağırandaki nesnenin bir *kopyası* olduğu anlamına gelir; fonksiyon çağrıldığı anda derleyici çağırandaki nesnenin alanlarını tamamen ayrı, yeni bir nesneye kopyalar. Gövde içindeki `n.x` değişikliği yalnızca bu kopyayı etkiler; fonksiyon geri döndüğünde kopya (ve üzerindeki değişiklik) birlikte yok olur, çağırandaki orijinal nesneye hiçbir zaman dokunulmamıştır. `void f(Nokta&`

`n)` imzasında ise `&` işareti `n`'yi çağırandaki nesneye doğrudan bir takma ad yapar — kopya hiç oluşmaz, `n` ile çağırandaki nesne bellekte aynı yeri gösteren tek bir varlıktır. Bu yüzden gövde içindeki `n.x` değişikliği doğrudan çağırandaki nesneyi değiştirir ve fonksiyon geri döndükten sonra da kalıcı kalır. Kısacası fark kopyanın olup olmamasıdır: değere geçmede kopya vardır ve orijinalden bağımsızdır, referansla geçmede kopya yoktur ve ikinci bir isimden ibarettir.

Soru (Ünite 3 — karşılık gelen `delete` hiç çağrılmazsa). `new` ile öbekte ayrılan bir nesne için karşılık gelen `delete` hiç çağrılmazsa bellek sızıntısı oluşur: o bellek program sonlanana kadar geri alınamaz, işaretçi kapsam dışına çıksa bile. Bunun nedeni öbeğin — yığının aksine — kapsamla otomatik ilişkili olmamasıdır; bir işaretçi değişkeni kapsam dışına çıktığında yalnızca *işaretçinin kendisi* (adresi tutan değişken) yok olur, işaretçinin gösterdiği öbek belleği hiç etkilenmez. Eğer o adresi saklayan başka hiçbir değişken kalmadıysa — örneğin işaretçi ikinci bir `new` çağrısının sonucuyla üzerine yazıldıysa ya da hiç `delete` edilmeden fonksiyondan çıkıldıysa — o bellek parçasına bir daha asla erişilemez, dolayısıyla asla `delete` de edilemez; işletim sistemi o belleği ancak program tamamen sonlandığında geri kazanır.

Soru (Ünite 4 — elle yönetimin en büyük riski ve RAII'nin azaltması). Bir kaynağı elle `new`/`delete` ile yönetmenin en büyük riski, doğruluğun tamamen programcının disiplinine bağlı kalmasıdır: erken bir `return`, atlanan bir kod yolu ya da yalnızca bir satırı unutmak, tek başına bir sızıntıya yol açmaya yeter — kod tabanı büyüdükçe, ayrılma ile bırakılma arasındaki mesafe (satır, dosya, fonksiyon sayısı) büyür ve bu riski orantısız biçimde artırır. RAII (ilk bakış düzeyinde) bu riski, kaynağın bırakılmasını elle hatırlamanız gereken ayrı bir satır olmaktan çıkarıp, bir nesnenin kapsam sonunda otomatik çalışan özel bir fonksiyona bağlayarak azaltır: `IntTampon` örneğinde olduğu gibi, kaynağı yöneten `new`/`delete` çifti hâlâ vardır, ama artık sınıfın tanımı içinde tek bir yerde durur ve nesne kapsam dışına çıktığı her durumda — normal akışta da, erken bir `return`'de de — otomatik çalışır; siz her kullanım noktasında ayrıca "bırakmayı unutmayayım" diye düşünmek zorunda kalmazsınız.